Code Lab 1

Burcu Erarslanoglu (5948355)

EE4C12: Machine Learning for Electrical Engineering Applications
Delft University of Technology

September 11, 2024

1 Part A: Numpy Basics

Question 1

```
Person A: [37.221, -36.3014, -35.4, -40.9004]
```

Question 2

```
Absolute person B: [34.8698 34.3005 33.7987 39.0423]
```

Question 3

```
Round up C:[-40. -42. -35. -35.]
Round down D:[-35. -41. -43. -46.]
```

Question 4

```
Array CD:[[-40.8702 -42.6151 -35.8469 -35.2198]
          [-34.508  -40.1056 -42.3942 -45.5277]]
Shape of CD:(2, 4)
Sum of CD:[-75.3782 -82.7207 -78.2411 -80.7475]
```

Question 5

DopNet dictionary is used to produce the following result:

```
Person D:  [-34.508, -40.1056, -42.3942, -45.5277]
```

Question 6

Inside the arrange() function the values (1, 24+t, t) are written where t=0.5 and prod() is used to multiply all elements in the vector, giving the following results:

```
Vector:  [ 1.   1.5  2.   2.5  3.   3.5  4.   4.5  5.   5.5  6.   6.5  7.   7.5   8.
     8.5  9.   9.5 10.  10.5 11.  11.5 12.  12.5 13.  13.5 14.  14.5   15.  15.5 16.
    16.5 17.  17.5 18.  18.5 19.  19.5 20.  20.5   21.  21.5   22.  22.5 23.  23.5 24.
    ]
Multiplication of the elements in the vector:  8.820617546615546e+46
```

Question 7

Dot and cross products of $\mathbf{a} = [3, 5, 6]$ and $\mathbf{b} = [1, 2, 8]$ are as follows:

```
Dot product of a and b: 61
Cross product of a and b: [ 28 -18   1]
```

As can be seen above the dot product is a scalar quantity whereas the cross product is a vector quantity. The dot product represents the projection of vector **a** to vector **b**. It is formally written as $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos(\theta)$. The cross product represents the vector that is perpendicular to both **a** and **b** and it is expressed as $\mathbf{a} \times \mathbf{b} = |\mathbf{a}||\mathbf{b}| \sin(\theta)\hat{n}$. In both equations θ refers to the angle in between vectors. These give the geometric definitions of the dot and cross products.

Question 8

```
1 W1: [0, 2, 3]
2 W2: [0, 2, 3]
3 W1: [0, 2, 3]
4 W2: [0, 2, 3]
5 W3: [0, 2, 0]
```

In the first step, W2 was assigned as W1 with "W2=W1"; as a result, W1 and W2 refer to the same list. Therefore, when the first element of W2 is changed from 1 to 0, this also affects W1, since they point to the same list. The outputs for this part are in lines 1 and 2 above.

In the second step, W3 is created using the "copy()" function from W1 and then W3's last element is changed from 3 to 0. This modification only affects W3 and not W1 and W2 because W3 is essentially an independent copy of W1. The outputs for this part are in lines 3 to 5 above.

All in all, assigning a list directly to another one via "=" creates a reference to the same list, whereas the "copy()" function creates an independent and new list.

2 Part B: Statistics

Question 1

```
Max:  -26.377522754047824
Min:  -46.355691671939425
Mean:  -34.06158363575621
Median:  -33.65093668159305
Std:  4.322823033393506
```

Question 2

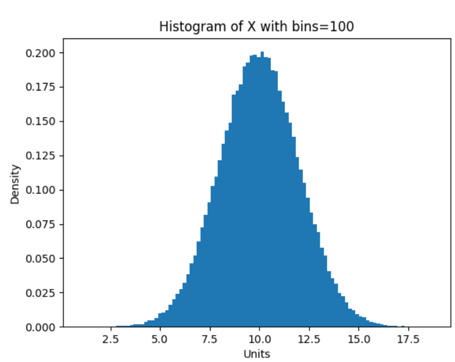


Figure 1: Question 2.

This part is also tried for various numbers of bins of the hist() function (the results are not put in because it was not asked). As bins determine how the data is grouped into intervals, changing the bins also changes how the histogram looks. When more bins are used, the data is divided into smaller intervals so more detail can be observed. When fewer bins are used, because of the larger intervals, the details are not that much but a general idea of the data can still be obtained.

Question 3

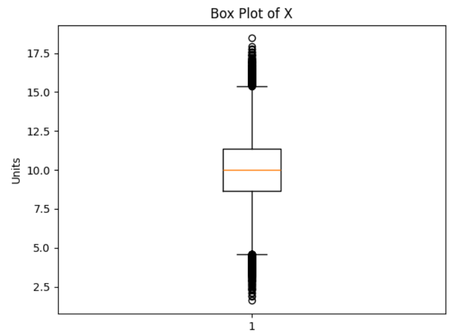


Figure 2: Question 3.

The boxplot gives us information about the median (shown with the orange line), the interquartile range (shown as the box), the whiskers that extend on the box to show the smallest and largest values within a specified range (shown as the black vertical line) and the outliers that are outside of the whiskers to show high and low values (shown as the circle dots).

Comparison: The comparison of the histogram and boxplot of X will now be given. The histogram of variable X shows a normal distribution, which peaks around the mean which is approximately 10. It has a symmetric bell shape where most of the values are clustered around the center. The density of the data decreases as it moves away from the mean. In contrast, the box plot of the same variable X summarizes the distribution by showing the median (again around 10) and the interquartile range. Different from the histogram, the box plot highlights outliers at both ends of the distribution. While the histogram provides more insight into the frequency of data points, the box plot displays central tendency, variability, and outliers. Both visualizations indicate a symmetrical distribution.

Question 4

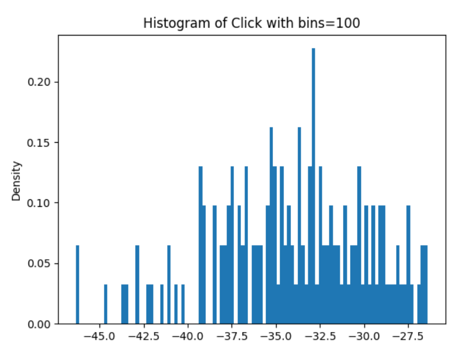


Figure 3: Question 4.

The histogram of the Click data is also shown for different bins and the results are shown below:

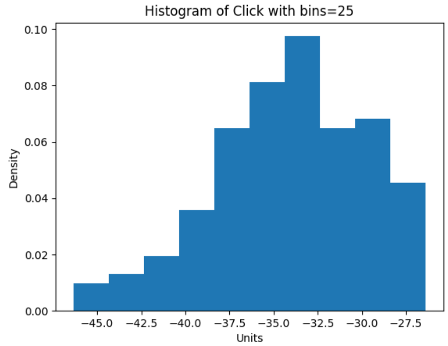


Figure 4: Bins=25

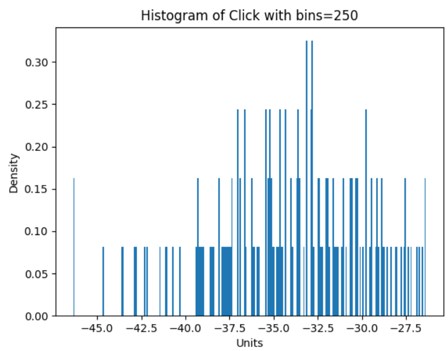


Figure 5: Bins=250

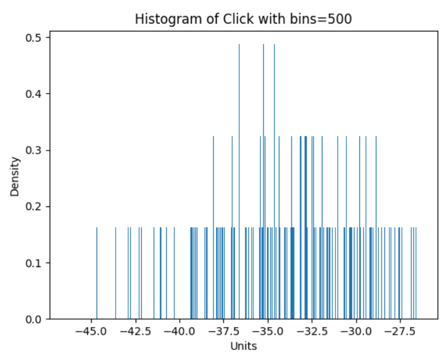


Figure 6: Bins=500

As can be seen, when the bin number is increased the data is divided into smaller intervals revealing much more detail. The fine details and variations in the data can be viewed. When the bin number is decreased the data is divided into larger intervals revealing less detail. Although the histogram shape becomes much simpler, important details get lost due to over-smoothing.

Question 5

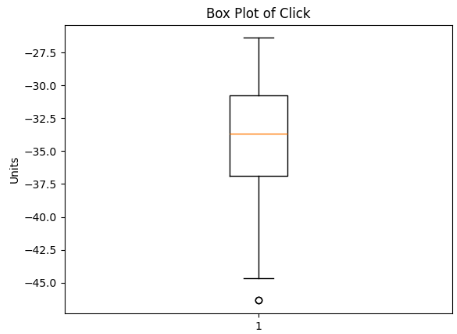


Figure 7: Question 5.

Comparison: The comparison of the histogram and boxplot of Click will now be given. The histogram of variable Click shows a less smooth and irregular distribution when compared to the one of X. The data is concentrated densely around the values between -37.5 and -30 and the distribution is uneven with some spikes and gaps in between those. In contrast, the box plot provides a more simplified summary, showing that the median is around -33.5 and has a wider interquartile range than the previous data X. The box plot also highlights one lower outlier below -45, which is not as easily distinguishable in the histogram in all of the histograms with different bins. Overall, the histogram provides a more detailed look at the irregular distribution of Click across the range, while the box plot offers a cleaner overview of central tendency and dispersion, with one clear outlier at the lower end at around -45.

Question 6

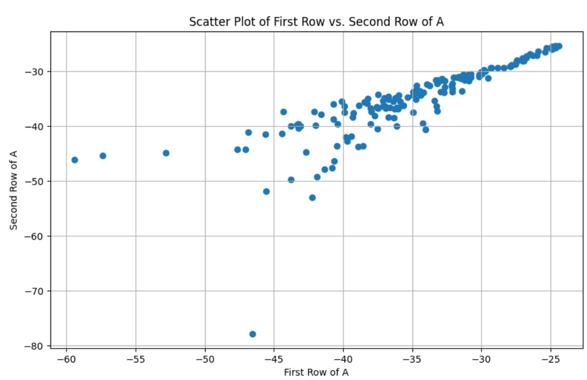


Figure 8: Question 6.

3 Part C: Loops (for, while, do while) & If/Else statements

Question 1

The pattern is followed to obtain the following elements:

```
[-30.02243593345215, -29.346431886462106, -30.33548646493486,
-32.62667369655402, -34.12127671198736, -33.15698735808775,
-35.43463365723958, -48.733732762956706, -40.99852625211449,
-36.389457522475645]
```

Question 2

For the first row of the main data of ‘personAclick‘, all the elements are negative as shown below:

```
positive: 0
negative: 154
zero: 0
```

4 Part D: Data manipulation (Introduction to Pandas)

Question 1

In this part, dataframe.size() function is used to return the number of all elements in the DataFrame. This is basically number of rows $\times$ number of columns. The results are as follows:

```
Dimensions of the data frame Click: 123200
Dimensions of the data frame Pinch: 94400
Dimensions of the data frame Swipe: 209600
Dimensions of the data frame Wave: 108800
```

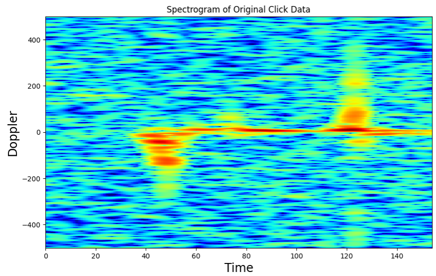
Question 2

For this part, the instruction given in the PDF file is followed where every 8^{th} column and every 12^{th} row is selected. In the ibynb file, the instructions are different. Because the indexing starts from 0 in Python, the 7^{th} column is first picked and stepped by 8, the same logic applies to the rows where the 11^{th} row is first picked and stepped by 12.

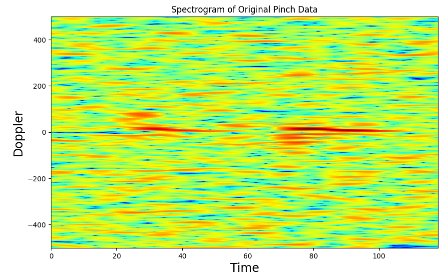
```
Value of the last row and column of Click: -28.908302170898835
Value of the last row and column of Pinch: -33.23645902494554
Value of the last row and column of Swipe: -16.282236512486023
Value of the last row and column of Wave: -34.80692405607001
```

Question 3

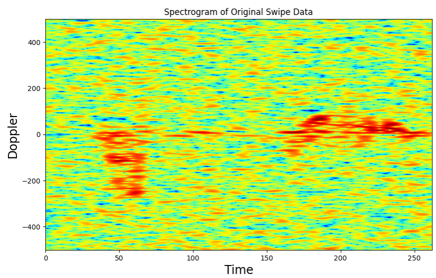
The necessary figures for this question are presented in the next page. When the downscaling of the data was carried out in Question 2, the idea was to reduce the resolution of the dataset by selecting only a subset of the original rows and columns. When the scaled-down spectrograms are compared to their original versions, it is seen that the key features in the data are retained in all Click, Pinch, Swipe, and Wave data. These features are dominant frequencies and general trends, so we may be able to deduce similar observations with the scaled-down spectrograms as well. Of course, some finer details are lost when some information in the data is disregarded. If we were to scale down more aggressively such as picking every 22^{th} column and every 15^{th} row, we would have lost more information and the spectrogram would probably not match the original as closely. The reasons for scaling down are related to efficiency because when you have fewer data, computations become faster. In large datasets, this scale-down is actually needed for practical analysis. Also, sometimes simplification might be needed to focus on the most important trends rather than details.



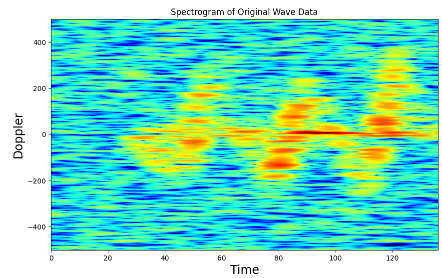
(a) Spectrogram of Original Click Data



(b) Spectrogram of Original Pinch Data

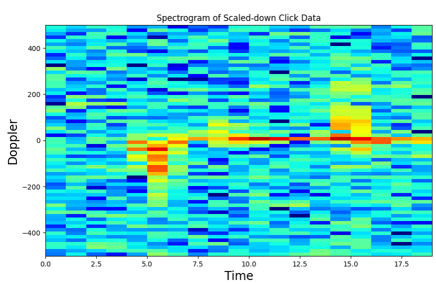


(c) Spectrogram of Original Swipe Data

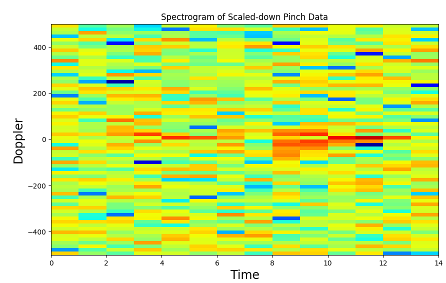


(d) Spectrogram of Original Wave Data

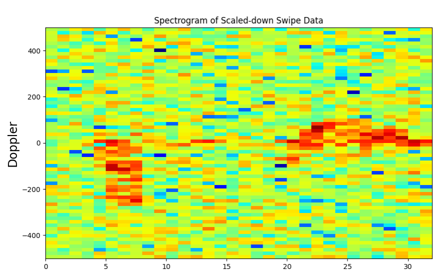
Figure 9: Spectrograms of Original Data for Click, Pinch, Swipe, and Wave



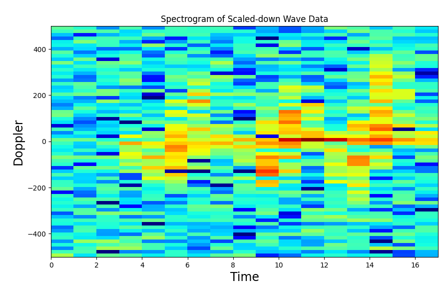
(a) Spectrogram of Scaled-down Click Data



(b) Spectrogram of Scaled-down Pinch Data



(c) Spectrogram of Scaled-down Swipe Data



(d) Spectrogram of Scaled-down Wave Data

Figure 10: Spectrograms of Scaled-down Data for Click, Pinch, Swipe, and Wave

5 Part E: Train-Test Split

The function written for this part where the DataFrame is shuffled in-place and index is reseted is presented below:

```python
# Function
def My_split(data, ratio):
    # if ratio not in the bound then give error
    if ratio < 0.0 or ratio > 1.0:
        raise ValueError("Ratio must be between 0.0 and 1.0")

    # shuffle the data
    shuffled_data = data.sample(frac=1).reset_index(drop=True)

    # determine the split index
    split_index = int(ratio * len(shuffled_data))

    # split the data according to the split index
    train = shuffled_data[:split_index]
    test = shuffled_data[split_index:]

    return train, test
```

The dimensions of train and test data frames are presented below:

```
Training set dimensions: (640, 154)
Test set dimensions: (160, 154)
```

Therefore, the training set has 640 samples and 154 features, and the testing set has 160 samples and 154 features.

This process is needed in machine learning for model evaluation. The training set is used to teach the model, while the test set assesses how the model works for unseen data. This process helps to prevent overfitting because the model can perform well for the memorized data but poorly on the new data. By using a separate test set, we can check if the model is truly learning the patterns in the data rather than simply memorizing (overfitting to the training data). It also ensures that the model is evaluated fairly and this provides robustness in real-world scenarios. The test set essentially simulates different scenarios by providing an independent dataset.

6 Bonus: Stratified Train-Test Split

```
Training set ratio of 0s to 1s:
 label
1    0.603125
0    0.396875
Name: proportion, dtype: float64
Test set ratio of 0s to 1s:
 label
1    0.5875
0    0.4125
Name: proportion, dtype: float64
```

As it was given in the instruction manual, the ratio of 0s and 1s still remains to be around 40% and 60%; even though the data has been split into training and testing sets. This is because the stratification ensures that each subset (training and test sets) maintains the same proportion of different categories or labels (0s and 1s) as in the original dataset. When the ratio is the same across the training and test sets, it is ensured that both sets are representative of the true distribution in the original dataset. It also avoids bias because a test set with too many 1s could perform better on a dataset that is dominated by the label 1. Overall, it aims for a more balanced and fair model evaluation.